A Comparative Study of Shared-Memory, GPU, and Distributed Parallelisation Strategies for 2D Image Convolution

Saatvik Das · Saket Rama · Zaid Khan

Department of Computer Science and Engineering, Manipal Institute of Technology, Bengaluru · Guide: Prof. Anil Kumar · PCAP, Semester V

728 / 728
runs match the serial reference (max error 4.8×10^{-7})

8.5×
best OpenMP speedup, 16 threads; flat after about 4

389×
GPU speedup counting only the filter kernel

104×
the same job including copies to and from the GPU

70–93%
of GPU job time spent on data transfer

86×
less cache traffic with shared-memory tiling (K = 15)

Why this study

Convolution is the sliding-window filter behind blur, sharpening and edge detection and behind the layers of convolutional neural networks. Every output pixel is independent, so it parallelises easily, yet it is limited by memory bandwidth, not arithmetic. Published comparisons usually use one programming model, tiny filters, and kernel time only, which flatters the GPU.

We ask: how much does each optimisation really buy, why, and what does the GPU cost end to end?

Aim

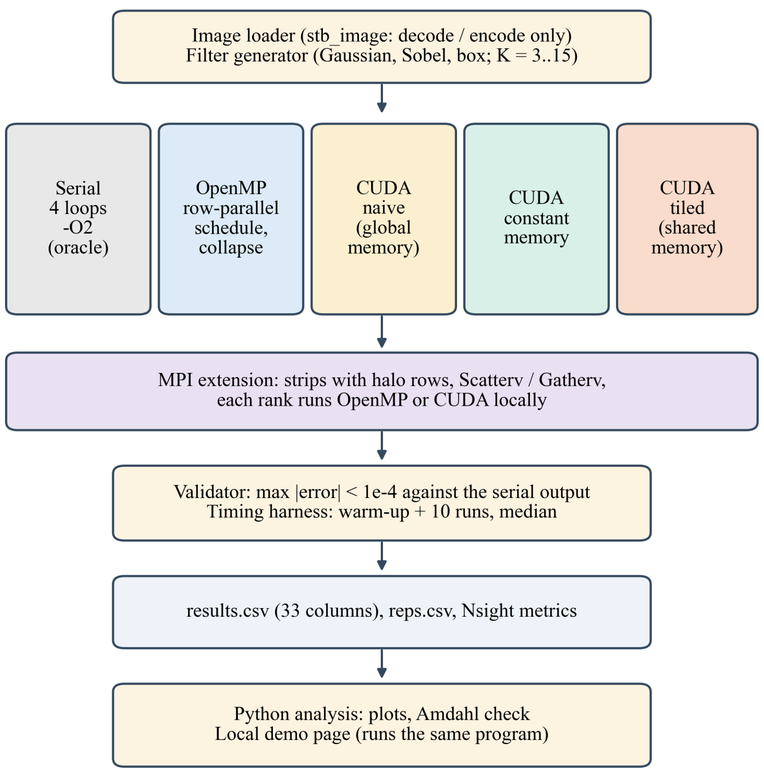
- Run **one** algorithm on five execution paths under identical input, validation and timing.
- Explain each speed change by a specific hardware mechanism, with profiler evidence.
- Count host–device transfers, not just kernel time.
- Compare measured speedup ceilings with Amdahl's law [1].

Algorithm

For a K×K filter F (K odd, R = (K−1)/2) and image I:

$$O[y][x] = \Sigma_{i,j=-R..R}\ I[y+i][x+j]\cdot F[i+R][j+R]$$

Single-precision float, zero-padded border. The plain four-loop serial code is the oracle: every other output must match it to within 10^{-4}.



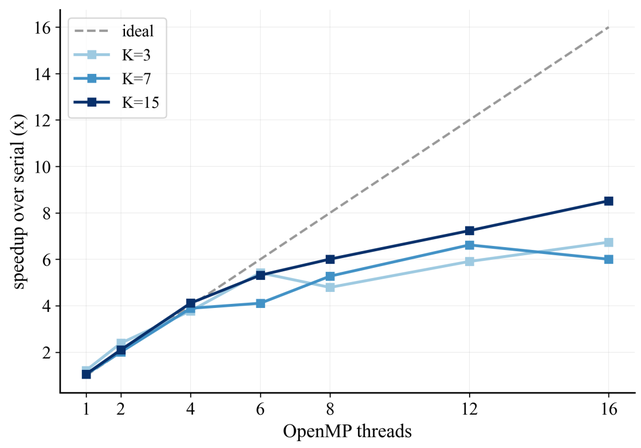
One shared loader, validator and timing harness feed all execution paths and the analysis scripts.

CUDA stages: *naive* reads everything from global memory; *constant* keeps the filter in constant memory (warp-wide broadcast); *tiled* also stages an input tile with halo in shared memory [7]. The serial code is compiled `-O2` without vectorisation; OpenMP shares rows between threads.

Method

- **Machine:** Intel i7-12650H (6P + 4E cores, 16 threads), RTX 3050 Laptop GPU (4 GB), CUDA 12.6, GCC 13.3, WSL2.
- **Sweep:** 4 images (two 4K, two 8K) × K = 3–15 × 26 configurations = 728 runs.
- **Timing:** one warm-up, 10 runs, median; CUDA events for kernel and each copy.
- **Check:** every output compared with serial; Nsight Compute for DRAM, L1 and occupancy.

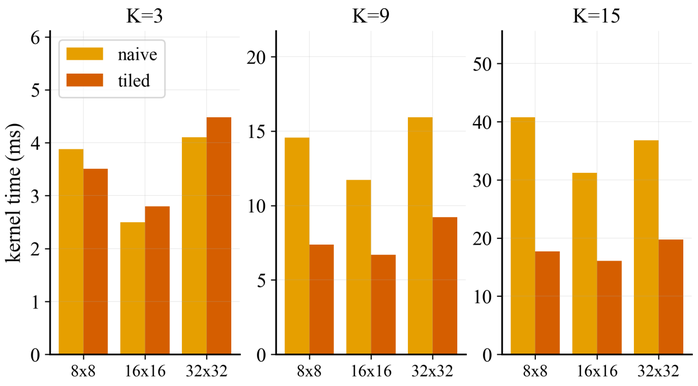
OpenMP stops scaling near 4 threads



Speedup over serial, 8K image. Ideal is the dashed line.

6.7×–8.5× at 16 threads (efficiency 0.38–0.53). Memory bandwidth and the hybrid P/E-core mix limit it; small filters saturate sooner. Dynamic and guided schedules beat static by 9–12%, and `collapse(2)` with chunk 1 is 5× slower.

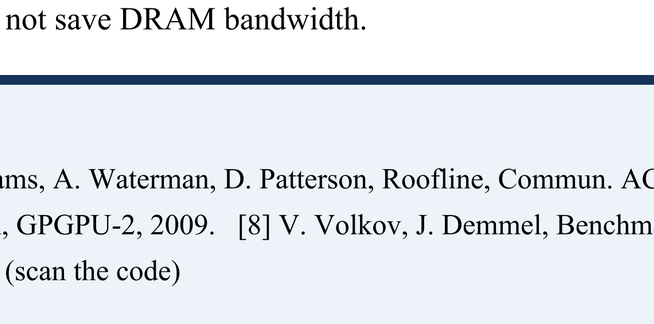
16×16 blocks are fastest



CUDA kernel time by block shape, 8K image.

Small blocks hold too few warps; 1024-thread blocks limit how many fit on a multiprocessor. This echoes the finding that maximum occupancy is not maximum speed [8].

Tiling pays off only for large filters



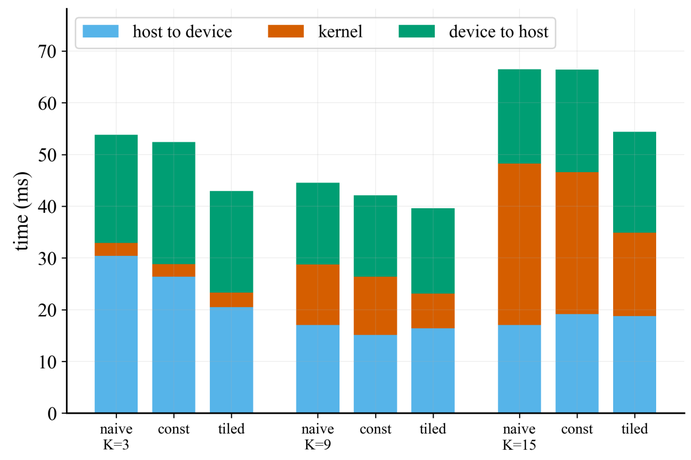
Kernel time of the naive kernel divided by the optimised kernel, 16×16 blocks.

The ratio grows from **0.89** at K = 3 (tiling slightly slower) to **1.94** at K = 15.

K	Kernel	DRAM read MB	L1 loads MB	Warps active
15	naive	137.9	51,192	88%
15	tiled	147.6	597	97%

Nsight finding: DRAM traffic is the same for all kernels (about one pass over the 133 MB image) because L1 and L2 already capture the reuse. Tiling removes **cache-to-multiprocessor** traffic and raises occupancy; it does not save DRAM bandwidth.

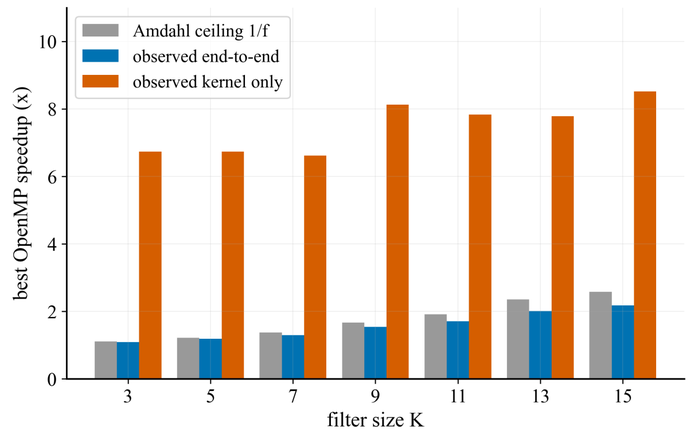
Transfers dominate the GPU's time



Host-to-device copy, kernel and device-to-host copy, 8K image, 16×16 blocks.

Copies are 93% of the tiled job at K = 3 and 70% at K = 15. Kernel-only speedup is 165× and 389×; whole-job speedup is **9× and 104×**. The GPU still beats 16 threads at every K, but offload pays only when there is enough work per byte moved.

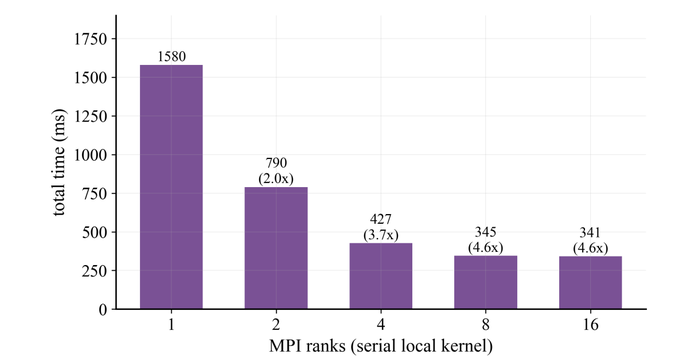
Amdahl's law against measurement



Ceiling from the measured serial fraction against the best observed OpenMP speedups.

Image decode and PNG encode make the serial fraction f = 0.90 (K = 3) to 0.39 (K = 15), so whole-job speedup is capped at **1.1×–2.6×**; the observed values are within 2–16% of that bound. The law misses the kernel-only gap (6.6×–8.5× instead of 16×): bandwidth contention and hybrid cores.

MPI extension



Strip decomposition on one node, serial local kernel, 8K image, K = 7.

About 5× at 16 ranks, then flat. Results match serial for 1, 2, 3, 4 and 7 ranks, including uneven row counts. Single-node runs show correctness and halo handling, not network scaling.

Conclusions

- Memory behaviour, not thread count, sets performance.
- Caches already absorb most reuse, so tiling helps through cache traffic and occupancy, and only for large K.
- Report end-to-end time: 389× became 104×, and about 8× at small K.
- Limits: one machine, synthetic grayscale images, single-node MPI.

References

[1] G. M. Amdahl, AFIPS Spring Joint Computer Conf., 1967. [3] S. Williams, A. Waterman, D. Patterson, Roofline, Commun. ACM 52(4), 2009.

[7] P. Micikevicius, 3D finite difference computation on GPUs using CUDA, GPGPU-2, 2009. [8] V. Volkov, J. Demmel, Benchmarking GPUs to tune dense linear algebra, SC '08.

Source code, results and full report: https://github.com/PCAP-ZSS/ConvoX (scan the code)